ParkEase: A Cloud-Based Parking Reservation System with Machine Learning-Based Demand Prediction

[Author Name 1], [Author Name 2], [Author Name 3]
[Department], [College / University]
[City, Country] · [email@example.edu]

***Abstract*—Finding a parking space wastes time and fuel, and small parking operators rarely know when their lots will be full. This paper presents ParkEase, a web-based parking reservation system built with React, Node.js/Express and MongoDB Atlas, designed for serverless deployment on AWS Lambda and Amazon API Gateway. The system models each reserved hour of a parking spot as a separate document protected by unique compound indexes, and writes a booking together with its hourly segments in one multi-document ACID transaction. In an experiment with 50 concurrent clients over 20 rounds, a naive check-then-insert approach double-booked a spot in all 20 rounds, while the proposed design admitted exactly one booking per round. A compound index reduced the documents examined by the availability query from 200,000 to 38. A Random Forest classifier, trained with scikit-learn on a clearly labelled synthetic occupancy history, predicts low, medium or high demand per lot and hour; on a held-out later time period it reached 86.2% accuracy and 0.799 macro F1, against 63.3% accuracy for a majority-class baseline. The model is exported to JSON and served inside the existing API with identical predictions, and is kept separate from booking validation. The estimated AWS cost for a demonstration workload is about US$0.03 per month. Limitations, chiefly the synthetic training data, are discussed.**

***Index Terms*—smart parking, reservation system, MongoDB, transactions, concurrency control, serverless computing, AWS Lambda, random forest, demand prediction.**

I. Introduction

Drivers in busy areas spend a noticeable share of their trips searching for parking. Shoup estimated from a series of studies that cruising for curb parking accounts for a substantial share of traffic in congested downtowns [1], and surveys of smart-parking research show a steady move from sensor-only systems towards reservation and prediction services [2]. For a driver, the most useful guarantee is a reserved spot; for an operator, the most useful information is when demand will peak.

ParkEase began as a student web application that let users browse parking lots and reserve spots. An inspection of the original code found a sound core (a transactional booking design) but also a fake spot map, static availability figures, short random booking references that could collide, no machine-learning component, and no automated tests. This paper describes how the system was completed into a working reservation platform that demonstrates three areas of a computer-science curriculum: database management, cloud computing and artificial intelligence.

A. Problem Statement

A parking reservation service must never sell the same spot twice for overlapping times, even when many users book at the same moment; it must show availability that matches the database; it must remain cheap enough to run on student cloud credits (about US$140 of AWS credit and no personal spending); and any demand forecast must inform users without ever overriding the actual booking state.

B. Objectives

1) Design a database schema whose constraints make overlapping and duplicate bookings impossible under concurrency, and demonstrate CRUD, indexing and aggregation.
2) Add a real machine-learning feature that predicts low, medium or high demand, evaluate it honestly and serve it through a secure API.
3) Design a low-cost serverless cloud deployment with monitoring, budget controls and secure configuration.
4) Verify the system with automated tests, including concurrent booking attempts, and report only measured results.

II. Related Work

Lin *et al.* survey smart-parking solutions across information collection, system deployment and service dissemination, and identify reservation and prediction as key services [2]. Zheng *et al.* predicted parking occupancy for sensor-enabled car parks in Melbourne and San Francisco and compared regression trees, neural networks and support vector regression, finding tree-based models competitive [3]. Random Forests, introduced by Breiman [4], are widely used for such tabular prediction because they capture non-linear interactions with little tuning; we use the scikit-learn implementation [5].

On the database side, the classic treatment of transactions and isolation is given by Gray and Reuter [6], and Berenson *et al.* show how weak isolation levels admit anomalies such as lost updates and write skew [7]. Kung and Robinson proposed optimistic concurrency control, which validates conflicts at commit time rather than locking in advance [8]; the approach taken here follows that spirit, using unique indexes as the validation step. MongoDB supports multi-document ACID transactions with snapshot isolation on replica sets [9]. For deployment, serverless computing lets applications scale to zero and bill per request [10], which suits workloads with low and irregular traffic such as a student project.

III. System Architecture

ParkEase has four parts (Fig. 1). A single-page React application is built with Vite and, in the AWS design, served from a private Amazon S3 bucket through Amazon CloudFront. The Express API contains authentication, parking-lot management, booking, administrative analytics and demand-prediction routes; it runs either as a long-lived Node.js process (locally and on the existing Render deployment) or as a single AWS Lambda function behind an API Gateway HTTP API using `serverless-http`. MongoDB Atlas stores all data. An offline Python pipeline trains the prediction model and exports it as a JSON file that is bundled with the API.

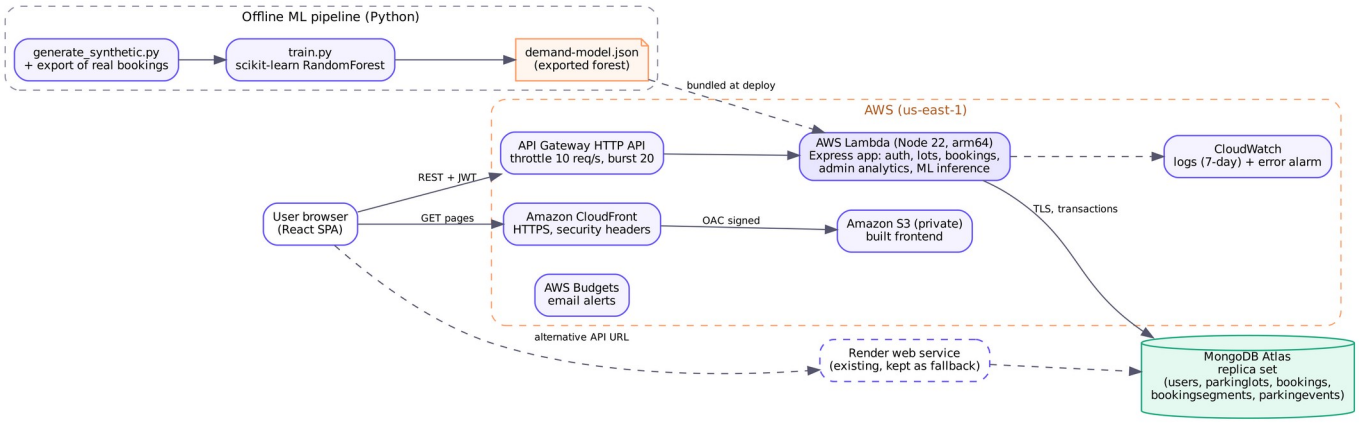


Fig. 1. System architecture. Solid arrows are request paths; dashed arrows are the existing Render fallback, logging and the offline model pipeline. The AWS components are designed and templated but not yet deployed.

The API is stateless: each request carries a JSON Web Token (JWT) and the user's role is re-read from the database on every request, so a role change takes effect immediately. This also allows any Lambda container to serve any request.

IV. Database Design and DBMS Implementation

A. Schema

The database has five collections (Fig. 2). `users`, `parkinglots` and `bookings` hold the main entities; bookings reference users and lots through `ObjectId` fields, playing the role of foreign keys. `bookingsegments` holds one document per reserved hour of a spot, and `parkingevents` is an audit log. The lot name is copied into each booking on purpose, as a historical snapshot for receipts. Spot identifiers (`A-01`, `A-02`, ...) are derived from a lot's capacity in rows of ten rather than stored, so they always match the capacity.

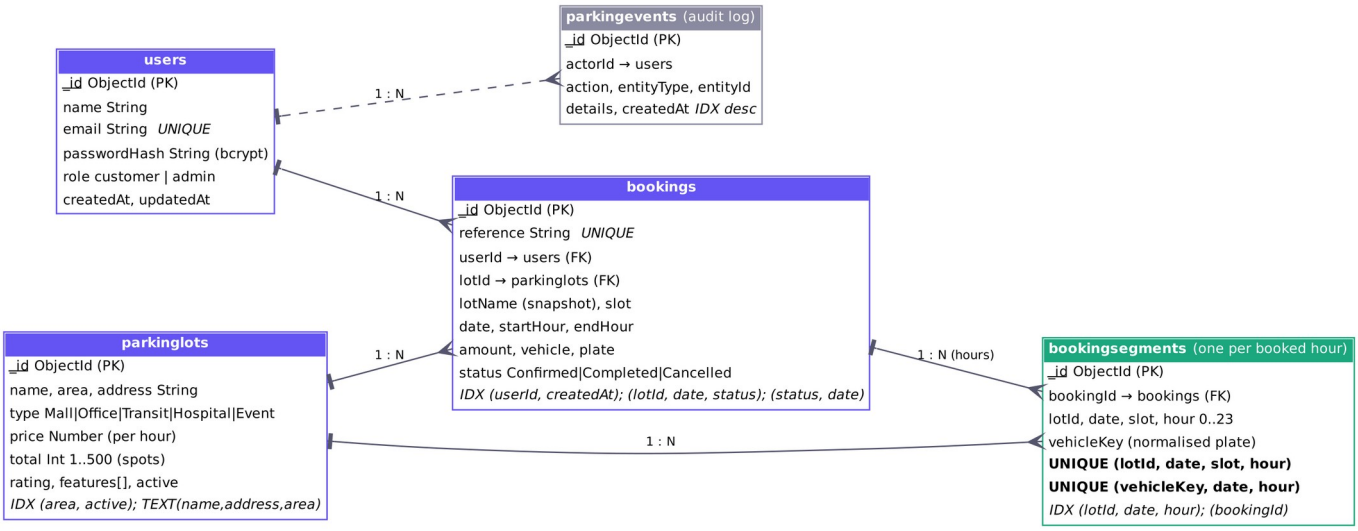


Fig. 2. Entity-relationship view of the MongoDB collections, with primary keys, references and the indexes that enforce integrity.

B. Preventing Overlapping and Duplicate Bookings

A booking from 09:00 to 12:00 creates one `bookings` document and three `bookingsegments` documents for hours 9, 10 and 11. Two unique compound indexes on the segments express the business rules as database constraints: `(lotId, date, slot, hour)` forbids two reservations of the same spot in the same hour, which is exactly the condition for two bookings to overlap; and `(vehicleKey, date, hour)`,

where `vehicleKey` is the normalised licence plate, forbids one vehicle from holding two spots at once and so blocks duplicate submissions. Booking references use six random base-32 characters per date, about one billion possibilities, and are also unique.

The booking document and all its segments are inserted in one multi-document transaction (Fig. 3). If any segment violates a unique index the whole transaction aborts, so a partially stored booking cannot exist (atomicity). Under snapshot isolation, two transactions that write the same index key conflict; the loser receives a transient `WriteConflict`, the driver retries it, and on retry it sees the committed segment and fails with a duplicate-key error that the API returns as HTTP 409. Commits use majority write concern on Atlas for durability. Cancellation likewise runs in a transaction that marks the booking cancelled and deletes its segments.

This is optimistic concurrency control in the sense of [8]: no locks are taken, and conflicts are detected at write time. It suits parking, where most concurrent requests target different spots. The common alternative, reading to check that a spot is free and then inserting, has a race window between the two steps; Section VII measures its effect.

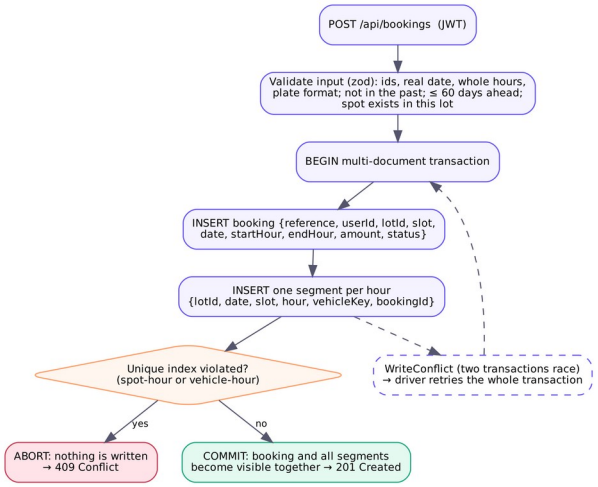


Fig. 3. Booking transaction. Validation happens before the transaction; the unique indexes decide conflicts inside it.

C. Other Integrity Rules

Mongoose schemas and MongoDB `$jsonSchema` validators enforce required fields, types, enumerations and ranges at the database level. The API additionally rejects bookings that start in the past (computed in the parking network's configured time zone), more than 60 days ahead, or for a spot outside the lot. A lot's capacity cannot be reduced below a spot with upcoming bookings, lots are archived rather than deleted so that old bookings keep a valid reference, and bookings whose end time has passed are marked `Completed` before bookings are read.

D. CRUD, Indexing and Aggregation

All four CRUD operations are exercised: users, lots, bookings and audit events are created; lots are listed, filtered and full-text searched; lots, profile names and booking status are updated; and segments are deleted on cancellation. Table I lists the indexes and the query each supports. Administrative figures are computed by aggregation pipelines: `$group` for totals by status and revenue, `$group` over segments for occupancy by hour of day, and `$lookup` (a join) from bookings to lots for revenue and average booking length per lot. An administrative endpoint returns the query plan of the availability query so that index use can be demonstrated live.

TABLE I
INDEXES AND THEIR PURPOSE

Collection	Index	Purpose
users	email (unique)	login; one account per email
parkinglots	(area, active); text	filters; search
bookings	reference (unique)	distinct references
bookings	(userId, createdAt)	booking history
bookings	(lotId, date, status)	upcoming bookings per lot
segments	(lotId, date, slot, hour) U	no overlapping bookings
segments	(vehicleKey, date, hour) U	one spot per vehicle
segments	(lotId, date, hour)	availability, live counts

U = unique. "segments" = bookingsegments.

V. Machine-Learning Demand Prediction

A. Task and Data

The model predicts, for a lot, date and hour, whether occupancy will be low (below 40%), medium (40% to below 75%) or high (75% or more). The thresholds are a design choice. Because the system is new, real bookings are far too few for training, so a simulator generates a **synthetic** history and every row is labelled as such. The simulator covers 12 lots (the six demonstration lots and six additional lots of other sizes and prices) for every hour of 2025, giving 105,120 rows. Its assumptions are explicit: hourly weekday and weekend occupancy profiles per lot type (office, mall, transit, hospital, event), a small price effect, seasonal effects, a random per-day effect and per-hour noise, and a binomial draw of occupied spots. A script can export real hourly occupancy from MongoDB with an aggregation, so real data can be added to training as it accumulates.

B. Features and Training

Features are hour, day of week, a weekend flag, month, capacity, hourly price and a one-hot lot type. The lot identifier is deliberately excluded so the model applies to new lots. Data are split by time: training on 2025-01-01 to 2025-10-31 (87,552 rows) and testing on 2025-11-01 to 2025-12-31 (17,568 rows), because a random split would place near-identical neighbouring hours in both sets. A Random Forest with 60 trees and balanced class weights was tuned by grid search over maximum depth {8, 12, 16} and minimum leaf size {5, 20} with three-fold time-series cross-validation scored by macro F1; the best setting was depth 16 and leaf size 20 (cross-validated macro F1 0.785).

C. Serving

Running scikit-learn in production would require a separate Python function or a hosted model endpoint, adding cost. Instead, the training script exports each tree as arrays of split features, thresholds, child indices and leaf class probabilities

(about 3.5 MB of JSON). The Node.js API walks each tree and averages the leaf probabilities, which is how a Random Forest computes class probabilities. A parity test on 300 held-out rows confirms that every predicted class matches scikit-learn and that probabilities differ by less than 0.0001. The prediction endpoint requires sign-in, validates its inputs and is rate-limited. It is never consulted when a booking is accepted or refused; the user interface labels it as a forecast and shows it beside, not instead of, live availability.

VI. Cloud Deployment, Security and Cost

A. Deployment Design

The AWS design (Fig. 1) uses one Lambda function (Node.js 22 on arm64, 512 MB) behind an API Gateway HTTP API with stage throttling of 10 requests per second and a burst of 20; a private S3 bucket and CloudFront with Origin Access Control and a managed security-headers policy; CloudWatch Logs with seven-day retention and an error alarm; and AWS Budgets email alerts. All resources are described in CloudFormation/SAM templates, which were validated with `cfn-lint`, so the whole deployment can be created and deleted as stacks. The Lambda handler connects to MongoDB once per container and reuses the connection across invocations. The previous Render deployment is preserved as a fallback.

B. Security

Passwords are hashed with bcrypt (cost 12). JWTs are signed with HS256 and verified with the algorithm pinned, which rejects unsigned tokens. Administrator rights are assigned only on the server. Every request body and query is validated with schemas that strip unknown keys, which blocks NoSQL operator injection such as `{"$ne": null}`. The API sets security headers with Helmet, allows cross-origin requests only from configured frontend origins, limits sign-in attempts to 20 per 15 minutes per client, limits prediction requests, caps request bodies at 20 KB and returns generic error messages. Secrets exist only in environment variables. During inspection, the original project archive was found to contain a real database password and token secret; these were removed from the code base and their rotation was recommended.

C. Cost Analysis

Table II estimates monthly cost for a demonstration load of 20,000 API calls of 300 ms each, using public AWS prices checked in October 2026 [11]-[13]. Lambda usage (3,000 GB-seconds) is inside the always-free monthly allowance of one million requests and 400,000 GB-seconds; the only billable item of note is API Gateway at US$1.00 per million requests. New AWS accounts now receive credits under a time-limited Free plan rather than the former 12-month free tier [14], and credits do not necessarily cover every charge, so the budget alert counts usage before credits are applied. The main financial risk is not this design but leaving costly resources (for example a NAT gateway or an always-on model endpoint) running; the guide therefore includes a clean-up procedure.

TABLE II
Estimated monthly AWS cost (demonstration load)

Service	Basis	US$/month
Lambda (arm64)	within always-free allowance	0.00
API Gateway HTTP API	20,000 × $1.00 per million	≈ 0.02
CloudFront	within 1 TB / 10M requests free	0.00
S3	≈ 2 MB stored	< 0.01
CloudWatch Logs + 1 alarm	within 5 GB / 10 alarms free	0.00
AWS Budgets	alerts are free	0.00
MongoDB Atlas	free cluster (not AWS)	0.00
Total		≈ 0.03

Estimate, not a measured bill. The AWS deployment has not yet been performed.

VII. Testing and Results

All results in this section were measured on 23 October 2026 on a four-vCPU Linux container using MongoDB 7.0 in Docker as a single-node replica set. None were measured on MongoDB Atlas or AWS; those tests are listed as future work.

A. Functional and Security Tests

An automated suite of 51 tests in four groups (unit, API, concurrency and rate limiting) runs against a real replica set; all 51 passed, and the concurrency group passed in three further repeated runs. Table III summarises coverage. A browser test using Playwright additionally registered a user, made a booking, confirmed that the spot then appeared as booked, and opened the administrator dashboard without console errors.

TABLE III
Automated test coverage (51 tests, all passed)

Area	Examples of checks
Authentication	validation, duplicate email, equal response for wrong password and unknown email
Tokens & roles	missing, forged, alg=none and expired JWTs rejected; 401/403 on admin routes
Input & errors	operator injection, bad ids, malformed and oversized JSON, impossible dates
Bookings	overlap refused, back-to-back allowed, one spot per vehicle, cancel rules
Concurrency	30 simultaneous requests: exactly one success for one spot
DBMS	aggregation totals consistent; availability query uses an index
AI	auth required, 24 hourly outputs, parity with scikit-learn, no effect on availability
Platform	security headers, CORS allow-list, sign-in rate limit

B. Concurrency Experiment

In each of 20 rounds, 50 clients attempted at the same moment to reserve the same spot and hour (Table IV). With check-then-insert, every round produced multiple "successful" bookings, up to 50 in one round. With unique indexes inside transactions, every round produced exactly one booking and no orphaned booking documents.

TABLE IV
Concurrent booking experiment

Approach	Rounds double-booked	Max winners	Total winners
Check then insert	20 / 20	50	539
ParkEase design	0 / 20	1	20

Ideal total winners = 20 (one per round).

C. Index Experiment

The availability query was run over 200,000 synthetic segments with a forced collection scan and with the `(lotId, date, hour)` index (Table V; median of five runs, timings from MongoDB execution statistics).

TABLE V
AVAILABILITY QUERY WITH AND WITHOUT AN INDEX

Plan	Docs examined	Keys examined	Median time
Collection scan	200,000	0	89 ms
Index scan	38	38	< 1 ms

Both plans returned 38 documents.

D. Model Evaluation

Table VI and Fig. 4 report results on the held-out test period. The forest reached 86.2% accuracy and 0.799 macro F1, compared with 63.3% accuracy and 0.258 macro F1 for always predicting the most frequent class. Errors occur almost only between adjacent classes; low and high demand were never confused. Medium demand is hardest (F1 0.634) because it is a narrow band whose hours are pushed across both thresholds by the simulated daily variation. Hour of day dominates the feature importances (0.62), followed by day of week (0.08) (Fig. 5). Because the data are synthetic, these figures show that the model recovers the simulated patterns; they are not evidence of real-world accuracy.

TABLE VI
DEMAND MODEL RESULTS ON THE TEST PERIOD

Class / model	Precision	Recall	F1	Support
low	0.950	0.934	0.942	11,112
medium	0.625	0.644	0.634	3,268
high	0.811	0.832	0.821	3,188
Random Forest (macro)			0.799	acc. 86.2%
Majority baseline (macro)			0.258	acc. 63.3%

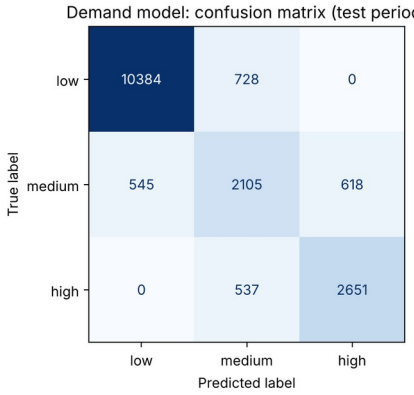


Fig. 4. Confusion matrix on the test period (rows actual, columns predicted). Diagonal: 10,384, 2,105 and 2,651 correct.

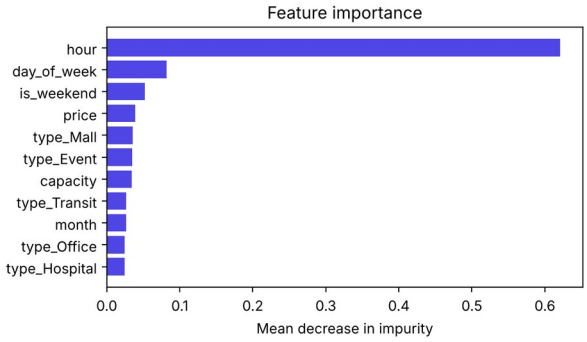


Fig. 5. Random Forest feature importance (mean decrease in impurity).

E. Latency

Table VII gives API latencies measured locally over 100 sequential calls per endpoint (rate limiting disabled for the measurement). A 24-hour forecast took 5.3 ms at the median, and a transactional booking 14.1 ms. Cloud latencies, including Lambda cold starts and the network path to Atlas, will be higher and have not been measured.

TABLE VII
LOCAL API LATENCY (MS)

Endpoint	p50	p95
GET /health	2	3.1
GET /lots	4.4	6.7
GET /lots/:id/availability	4	7.1
GET /predictions/demand (24 h)	5.3	7.6
POST /bookings (txn)	14.1	20.3

VIII. LIMITATIONS AND FUTURE ENHANCEMENTS

- The demand model is trained only on synthetic data; its real-world accuracy is unknown. Retraining on exported real occupancy is the most important next step.
- Features omit holidays, weather, local events and recent booking trends.
- The AWS deployment, and tests against Atlas and the cloud deployment, have been designed but not yet performed.
- Bookings use whole hours, and payment is not integrated.
- Application rate limits are per server instance; on AWS they are backed by API Gateway throttling.

Future work includes retraining with real data and drift monitoring, comparing gradient-boosted models, sensor or camera integration for occupancy, online payment, and demand-aware pricing.

IX. CONCLUSION

ParkEase shows that a small team can build a reservation system whose central guarantee, no double booking, is enforced by the database itself rather than by application checks. Modelling reserved hours as uniquely indexed documents written in a single transaction admitted exactly one booking in every round of a concurrency experiment in which a naive approach failed every round. A Random Forest demand forecast was integrated without new infrastructure by exporting the model to JSON, and is kept strictly separate from booking validation. A serverless AWS design keeps the expected cost of

a demonstration deployment to a few cents per month. The main open item is to replace synthetic training data with real occupancy history.

References

[1] D. C. Shoup, "Cruising for parking," *Transport Policy*, vol. 13, no. 6, pp. 479–486, 2006, doi: 10.1016/j.tranpol.2006.05.005.

[2] T. Lin, H. Rivano, and F. Le Mouël, "A survey of smart parking solutions," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 12, pp. 3229–3253, Dec. 2017, doi: 10.1109/TITS.2017.2685143.

[3] Y. Zheng, S. Rajasegarar, and C. Leckie, "Parking availability prediction for sensor-enabled car parks in smart cities," in *Proc. IEEE 10th Int. Conf. Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, Singapore, 2015, pp. 1–6.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

[7] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1995, pp. 1–10.

[8] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Trans. Database Syst.*, vol. 6, no. 2, pp. 213–226, 1981, doi: 10.1145/319566.319567.

[9] MongoDB, Inc., "Transactions," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/ (accessed Oct. 23, 2026).

[10] E. Jonas *et al.*, "Cloud programming simplified: A Berkeley view on serverless computing," Univ. California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019.

[11] Amazon Web Services, "Amazon API Gateway pricing." [Online]. Available: https://aws.amazon.com/api-gateway/pricing/ (accessed Oct. 23, 2026).

[12] Amazon Web Services, "Amazon CloudFront pricing." [Online]. Available: https://aws.amazon.com/cloudfront/pricing/ (accessed Oct. 23, 2026).

[13] Amazon Web Services, "Amazon CloudWatch pricing." [Online]. Available: https://aws.amazon.com/cloudwatch/pricing/ (accessed Oct. 23, 2026).

[14] Amazon Web Services, "AWS Free Tier update: New customers can get started and explore AWS with up to $200 in credits," AWS News Blog, Jul. 2025. [Online]. Available: https://aws.amazon.com/blogs/aws/aws-free-tier-update-new-customers-can-get-started-and-explore-aws-with-up-to-200-in-credits